

Introduction to Embedded Systems (WS25/26)

Niklas Rodenbüsch | February 9, 2026

1 Basics

1.1 Terminology & Definitions

Definition 1.1.1 (Embedded Systems)

Embedded System (ES) are information processing systems embedded into a larger (technical) product, controlling the larger system or providing information processing for it.

Definition 1.1.2 (Embedded Software)

Embedded software is computer software, written to control embedded systems. Typically specialized for the particular hardware that it runs on and has time and memory constraints.

Definition 1.1.3 (Cyber-Physical System (CPS))

Complex network of **real-time** interacting elements (e.g. ES) with physical input and output instead of standalone devices.

1.2 Applications

ES can be applied to a multitude of different domains, such as:

- **Automotives:** ABS, ESP, Airbags, ...
- **Aerospace:** Anti-Collision, Flap control, ...
- **Wearables:** Glasses, Watches, ...

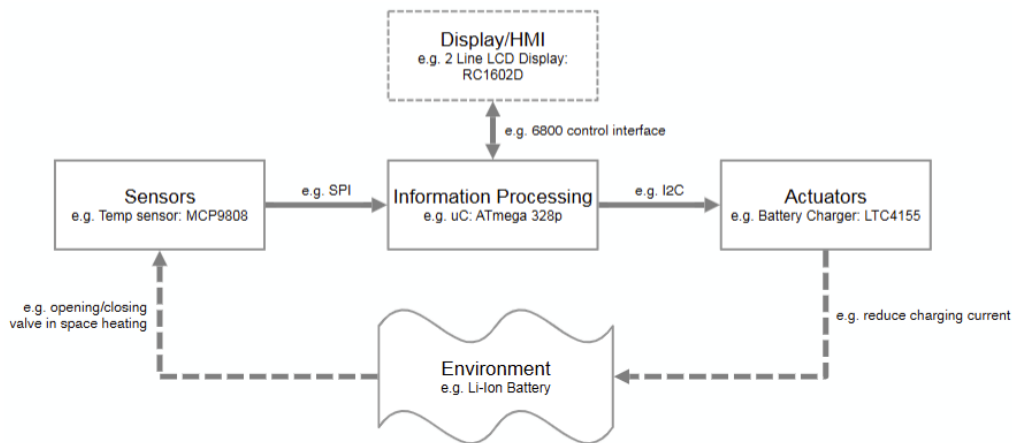
1.3 Characteristics

ES must be any number of the below characteristics (depending on the requirements, **not** all at once):

1. **Efficiency:** Energy, Area (Hardware), Code-size, Run-time, Weight, Cost
2. **Dependable:** Reliability $P(\text{system working correctly after working at } t = 0)$; Maintainability $P(\text{system working correctly } t \text{ time steps after error})$; Availability $P(\text{system working at } t)$; Safety; Security (confidential data & authentic communication)
3. **Real-Time capable:** System must react to stimuli within a given time interval; late reactions arriving too late are wrong; hard-constrained = if failure to meet the constraint results in catastrophe (else soft-constrained); response has to be guaranteed without statistical arguments
4. **Reactivity:** Superset of real-time constraint; behaviour of system depends on input and current state (can be modeled as automata); continuous interaction with environment (at a pace given by the environment) through sensors and actuators

1.4 Components / Feedback loop

The feedback loop typically contains: (1) Sensor(s), (2) Information Processing Unit, (3) Actuator(s), (4) Memory and the (5) Environment. Depending on the ES some of them are optional or simply not needed. Example below: battery charging system.



2 Specification

The first step of designing an ES is to translate the description into a formal specification, and **prove** that the specs meet the goals the system.

2.1 Abstraction/Hierarchy

- **Behavioural hierarchy:** Describe the systems overall behaviour (states, events, outputs)
- **Structural hierarchy:** Describe the physical composition (processors, actuators, sensors)

2.2 Timing Behaviour

Since ES are real-time systems, explicit timing requirements must be specified:

- (1) Elapsed time for execution;
- (2) Delay of processes;
- (3) Timeouts for events;
- (4) Deadlines for task finishing

2.3 Dependable Systems

Dependable systems fulfill the following requirements:

- (1) Unambiguous semantics;
- (2) Support formal verification;
- (3) Are executable;
- (4) Appropriate model for computation (execution model for each component, communication model)

2.4 Further Specifications

- **Event-Handling:** Specify how internal & external events are handled.
- **Concurrency:** Specify how distributed and parallel operations happen.
- **Synchronization and Communication:** Communication (i.e. mutual exclusion) between components
- **Non-functional properties:** Fault tolerance, Weight, Size, Power consumption, ...

2.5 Computational Specifications

- **Presence of programming elements** (i.e. arithmetic operations, loops, function calls)
- **Executability:** Plausibility checking
- **Readability**
- **Domain-Specific support**

3 Finite State Machine (FSM)

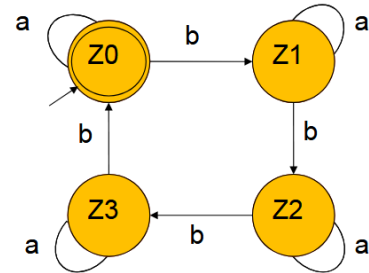
An FSM is a (graphical exception-oriented) model for the internal state of an embedded system (describes states, actions and how to handle exceptions). Inputs are internal/external events, and states are the memory content after events.

FSM can be used for e.g. input validation, concurrent event handling, or program flow (interrupt handling).

Definition 3.0.1 (FSM / Acceptor)

$M = (I, S, s_0, \delta, F)$ is a deterministic finite automaton, iff:

- I is a finite, non-empty set of input symbols
- S is a finite, non-empty set of states
- $s_0 \in S$ is the initial state
- $\delta : S \times I \rightarrow S$ as transition function
- $F \subseteq S$ are final (accepted) states



3.1 Moore and Mealy Machines

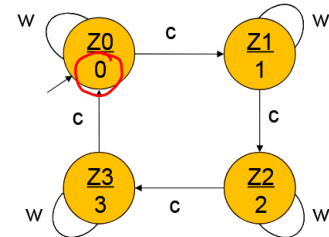
Definition 3.1.1 (Moore Machine)

Moore machines $(I, O, S, s_0, \delta, F, \lambda)$ extend acceptors by:

- O as a finite, non-empty set of output symbols
- $\lambda : S \rightarrow O$ as output function

The Moore Machine on the right outputs the result of $c\%4$.

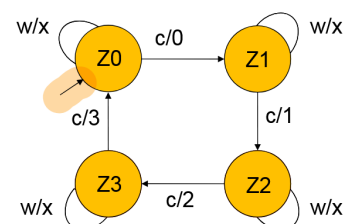
Note: For an input of length n , we get $n + 1$ outputs, as we get to s_0 using the empty input symbol $w_0 = \epsilon$.



Definition 3.1.2 (Mealy Machine)

Mealy machines $(I, O, S, s_0, \delta, F, \lambda)$ differ from Moore machines in their output function: $\lambda : S \times I \rightarrow O$

Mealy machines create outputs during state transitions, with the output for the initial transition being '-'. Examples: Enigma, Traffic light, Number classification, ...



Mealy machines depend on the current state and the current input symbol, while Moore machines can be seen as special Mealy machines where the output does not depend on the current input symbol.

Important: Moore can be transformed into Mealy and (partially) vice versa.

4 Statecharts

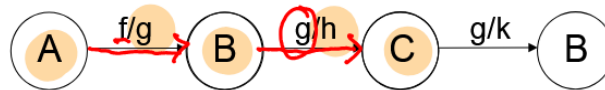
Especially when dealing with complex systems (many states) and/or concurrency, Moore and Mealy are not useful. Statecharts extend the classical machines with concepts like non-determinism (OR), history, variables, concurrency (AND), and timing/simulation.

Statecharts produce I/O differently as well: Transitions can be active on the presence of events, produce events, or change variables.

Real-world examples: Automotive applications (industry standard), UML

4.1 Transitions

A transition from state A to B occurs iff event f is present while being in A. On transition the event g is produced. Produced events exist for one step only.

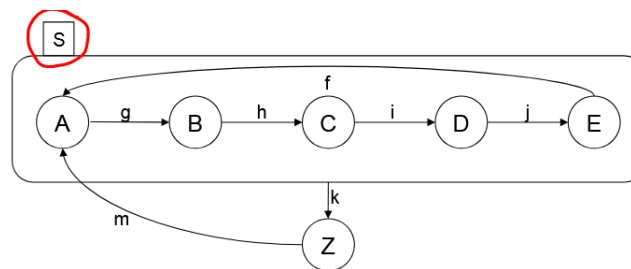


4.2 Non-Determinism

Multiple events can be present at the same time. While non-deterministic transitions are possible, tools usually warn beforehand so they can be handled using e.g. priority specification (see below).

4.3 Hierarchy

Statecharts introduce superstates S . Superstates S are called **OR**-superstates, if exactly one of the substates of S is active whenever S is active.



Current states of statecharts are called active states. States that are not composed of other states are called basic states. States containing other states are called superstates. For each basic state s , the superstates containing s are called ancestor states. When a basic state is active, all its ancestor states are active too.

The state hierarchy can be represented as a tree, where basic states are the leafs. The highest level in the hierarchy has priority when multiple transitions (i.e. events) are active at the same time.

4.4 History

To determine which substate is entered when a superstate is entered, different history mechanisms can be used. **Note:** Nested superstates need a default state each!

- **Default:** Filled circle (not a state itself) pointing to a substate. Entered each time S is entered.
- **History:** Enters substate S was in before S was left, or default if S entered for the first time.
- **Deep History:** Deep history remembers states at the sub-hierarchy level too!

4.5 Event-Condition-Action

Statecharts include typed variables (e.g. numbers) to represent data (states + variables = *status*). Values of variables keep their value until they are reassigned. Values of variables are visible to all parts of a statechart (shared memory). New values become effective in the execution stage for the current step and are obtained by all parts of the model in the *following* step.

This makes transition functions possible: A transition may be taken if an event occurred in the last step AND a condition referring to a variable is true. Actions can then either be variable assignments or creation of events.

Event types: Atomic events (A, B, BUTTON), entering/exiting state $en(S)/ex(S)$, variable value change $ch(x)$, access to variable $rd(x)/wr(x)$, condition value changes $tr(co)/fs(co)$

Condition types: Atomic conditions, constants (true/false), relations between values, residing in a state $in(S)$

Action types: Atomic actions, emitting events $raise(E)$, variable assignment, compound actions, list of actions, conditional actions

4.6 Concurrency

Concurrency can be expressed via **AND**-Superstates, where *all* substates are active when S is active.

Important: Concurrency can lead to different types of conflicts (e.g. write-write if two parallel transitions write the same variable).

4.7 Timers

Statecharts support special timeout events, e.g. "if event a does not happen while being in state X for 20ms, a timeout event will take place and activate state Y ".

4.8 Simulation

Execution of a statechart consists of a sequence of steps (arbitrarily small time), each leading from one *status* to another. At each step, the current system status s_i , the current time t , and internal/external changes Δ are given. The goal then is to find the internal events e_i and new status s_{i+1} .

External events not yet seen in the previous step and changes of external data not seen in the previous step are called external changes for the current step.

Status now consists of: active states, values of variables, generated events from previous steps, values of history connectors, set of all timeout events.

Simulation Step

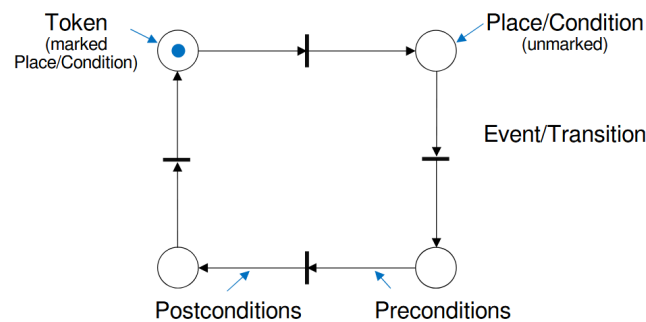
(1) Update timeout events $\rightarrow$ (2) Determine sequence of transitions to be taken, based on current i/e events and i/e variables $\rightarrow$ (3) Compute next states and actions $\rightarrow$ (4) Apply transitions

4.9 Pros and Cons

Advantages	Disadvantages
Simple transformation into hard- and software implementations	Complex for large systems
Efficient for simulation	Not useful for distributed applications
Basis for formal verification	No formal representations of operations and data
Arbitrary nesting of superstates possible	Large part of status is hidden in variables

5 Petri/CE/PT Nets

In contrast to hierarchical state machines, state transitions in Petri Nets are asynchronous. Focus on modelling causal dependencies; no global synchronization assumed $\rightarrow$ message passing only. Petri nets can be used to model concurrent distributed systems (e.g. UML).



5.1 Components

- **Conditions:** Represent "local states" and are either met or not met (potential state space)
- **Events:** May take place if certain conditions are met (represents state transition)
- **Flow relation:** Relates conditions and events
- **Tokens:** Assignment of tokens to conditions specify a global state

Definition 5.1.1 (Petri Net)

A Petri Net $N = (C, E, F, M_0)$ is defined with:

1. C : set of conditions/places
2. E : set of events/transitions
3. F : set of edges (binary flow relations) with $F \subseteq (C \times E) \cup (E \times C)$, C and E disjoint
4. $M_0 : C \rightarrow \mathbb{N}$ as the initial marking

Definition 5.1.2 (Preconditions / Postconditions)

Let N be a Petri Net and let $x \in E, y \in C$:

1. $\cdot x := \{y \mid (y \times x) \in F\}$ is called the set of preconditions
2. $x \cdot := \{y \mid (x \times y) \in F\}$ is called the set of postconditions

5.2 Marking

A marking is a mapping $M : C \rightarrow \{0, 1\}$. Firing event x generates new markings on all conditions C according to the following rules:

1. An event x can be fired, iff $\cdot x$ is marked and $x \cdot$ is unmarked
2. After firing, $\cdot x$ is unmarked and $x \cdot$ is marked
3. $M \rightarrow M'$, iff M' results from M by firing *exactly* one transition

5.3 Condition-Event Nets**Definition 5.3.1 (Different Concepts)**

1. $(c, e) \in C \times E \rightarrow (c, e)$ is called **loop** iff $cFe \wedge eFc$.
2. Net $N = (C, E, F)$ is called **pure**, if F does not contain loops.
3. Net is called **simple**, if there are no two transitions that have the same sets of input and output places: $\forall x, y \in E : \cdot x = \cdot y \wedge x \cdot = y \cdot \Rightarrow x = y$

Definition 5.3.2 (C/E-Nets)

A Petri Net $N = (C, E, F)$ together with a non-empty set of markings M is called C/E-Net, iff:

- N is simple, pure, and has no isolated elements
- M is closed wrt. "firing" and "inverse firing"
- N is deadlock-free: For each event $e \in E$ exists a marking $m \in M$ that allows firing at e

5.4 Properties

- Marking M' is **reachable** from marking M , iff there exists a sequence of firings that transform M into M'
- C/E net is **cyclic**, iff any two markings are reachable from each other
- C/E net fulfills **liveness**, iff N is deadlock-free
 - **Strong liveness:** For each M and e exists M' so that M' is reachable from M and M' activates e for firing.
 - **Weak liveness:** For each M an event e exists so that M activates e for firing.

5.5 Place/Transition Nets

PT-Nets can model resource (i.e. producer/consumer) usage and causal relationships, by allowing more than one token per condition, capacities of places, and weights of edges. The downside is that the state space may become infinite.

Nets can be used to verify properties such as reachability, liveness, or bounds on the number of tokens.

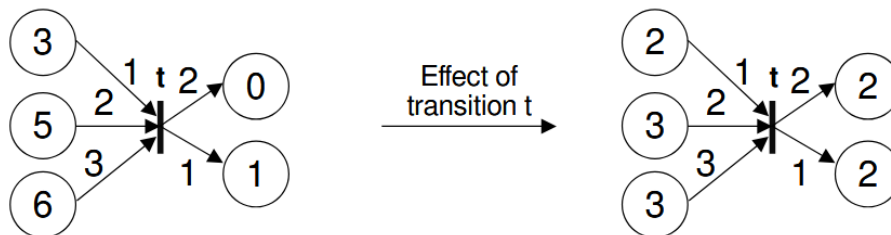
Definition 5.5.1 (PT-Nets)

A net $N = (P, T, F, K, W, M_0)$ is called PT-Net (non-deterministic), iff:

- $N = (P, T, F)$ is a net with places $p \in P$ and transitions $t \in T$
- $K : P \rightarrow (N_0 \cup \{\omega\}) \setminus \{0\}$ denotes the capacity of places; $\omega \equiv \infty$ capacity. A place p_i can hold $K(p_i)$ tokens at maximum.
- $W : F \rightarrow (N_0 \setminus \{0\})$ denotes the weight of graph edges
- $M_0 : P \rightarrow N_0 \cup \{\omega\}$ represents the initial marking of places

A net N is **contact-free**, if capacities in $\cdot t$ never prevent activation of the transition t . A **pure** and **contact-free** PT-Net is completely defined by N and M_0 .

Since capacities of places are not considered in N (here: matrix $P \times T$), we can not always compute transitions!



Pros and Cons

Advantages	Disadvantages
Can model distributed applications	problems with modeling timing
Well-known theory for proving properties	no programming elements
	no hierarchy

6 Design Space Exploration and Optimization

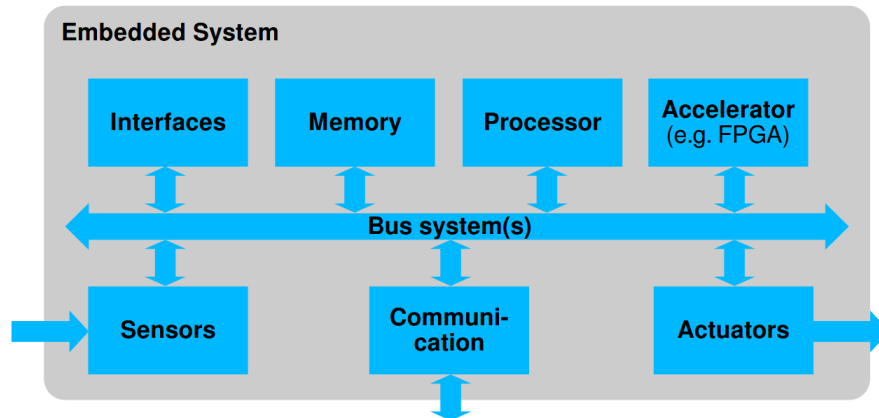
The design space of ES has many different parameters and aspects that can be optimized, thus yielding a multiobjective optimisation problem. Examples are:

Design Space: Cost, Performance, Energy, Quality, ...

Implementation: Application specific integrated circuitss (ASICs), memory architectures, interfaces (I²C, SPI, CAN), Sensors & Actuators, programmable devices (microcontrollers/-processors), ...

Processing: Clock frequency, OPS, Bandwidth, FPS, Worst-Case vs. Average-Case performance, ...

Individual elements of the design space can be further split up into multiple aspects, e.g. cost $\rightarrow$ manufacturing cost, design cost, administration, ...



6.1 Power Consumption

One of the most important constraints is power consumption! Minimising power consumption is important for the design of the power supply, the design of voltage regulators, dimensioning of interconnects, and short term cooling. Additionally, energy is expensive, batteries have limited capacity, and many other reasons.

Typical low-power design techniques include: (1) Low-power transistors (less speed), (2) dynamic power management (e.g. sleep modes), (3) dynamic voltage or frequency scaling.

6.2 Quality

Several aspects of quality must be taken into account when optimising cost:

- **Defect Level (DL):** Fraction of defective parts delivered to customer (measured in DPM)
- **Yield:** Fraction of defect-free parts
- **Reliability:** Probability that a part will work correctly during its lifetime (safety-critical)

6.3 Pareto-Frontier

Pareto-Frontier analysis can be used for multiple target optimization by quantifying the design targets (as parameter vectors):

Given two solutions $A = (a_1, \dots, a_m)$ and $B = (b_1, \dots, b_m)$, then A is dominating/pareto-superior to B iff $\forall j : a_j \geq b_j \wedge \exists i : a_i > b_i$.

7 Memory

Nowadays there are two types of machine models: more flexible "von Neumann" machines (which use one homogeneous memory for instructions and data), and more secure "harvard" machines which separate instruction and data memory.

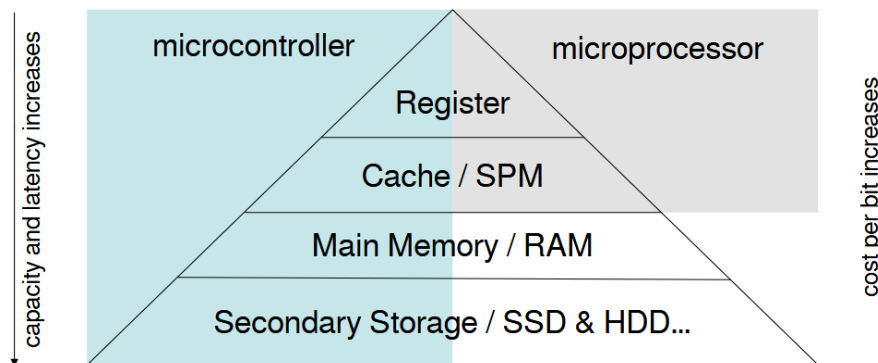
One of the biggest issues in modern computing is the "memory wall", which is a synonym for the fact that processor speed is (increasingly) far faster than memory access times.

7.1 Locality

Programs tend to use data and instructions with addresses near or equal to those they have used recently. Recently referenced items are likely to be referenced again in the near future (temporal locality). Items with nearby addresses tend to be referenced close together in time (spatial locality).

7.2 Memory Hierarchy

For efficiency, fast memory access times and large memory sizes are required. However, not all memory is required all the time. This results in a trade-off problem: Latency vs cost + energy.



7.3 Cache

Smaller, faster storage that acts as staging area for subset of data in a larger, slower device. Data is copied in block-sized units from main memory. Main question is which cache blocks to map the memory blocks to?

Direct Mapping: $[\text{Cache block address}] = [\text{Memory block address}] \bmod [\# \text{ cache blocks}]$

Associative Cache: k blocks are in cache associated with same index. Cache tags are used to link to a specific set of memory blocks by comparing tags. When cache is full, replace block using FIFO or LRU (least recently used).

Note: For the Worst Case Execution Time analysis one has to assume that (almost) every access to the cache results in a cache miss.

7.4 Memory Types

Non-volatile: Keeps memory state without power (e.g. Flash, EPROM), read-only, usually slow and limited in number of write cycles, stores programs and rarely data.

Volatile: Keeps state only for short time, SRAM for fast and deterministic access (but power hungry, used as Cache), DRAM which depends on access order and needs periodic refresh

7.5 Scratch Pad Memory (SPM)

SPMs are small physical memories directly mapped into the CPU's address space. This makes them fast and predictable and less power intensive. Usually frequently used instructions, variables, and intermediate results are stored in the SPM. SPM does not use tags, memory is directly accessed by the address.

8 Buses and Communication

8.1 I/O Access

Memory-mapped I/O: The components get mapped into the controller address space, enabling convenient transfers as writing to the memory address sends data to the component. Downside: May keep CPU busy and needs polling for updates.

Port-mapped I/O: Uses special CPU instruction to transfer bytes. Data moves from one component via a bus address to the memory address. Also keeps CPU busy and needs polling for updates.

8.2 Direct Memory Access (DMA)

DMA is the component used in most μC , it offloads I/O transfers from the CPU.

1. CPU instructs it to copy n bytes from the I/O device to a known location in the memory **directly**.
2. DMA signals the CPU when the transfer is complete. In the mean time, the CPU is free.

The usage of DMA gives rise to the issue of **cache coherency**:

- CPU accesses location X in memory, the current value will be stored in the cache
- Operations on X will update the cached copy of X , but not the external memory version
- If the cached copy of X is not invalidated when a device writes a new value to the memory, then the CPU will operate on a stale value of X

To mitigate this issue, e.g. simple 'invalidate' flags can be broadcasted onto the bus on every write.

8.3 Interrupt Requests (IRQs)

Interrupts trigger the CPU on events, e.g. when a DMA operation is finished, on a keyboard press, or on a timeout. The procedure works as follows:

- (1) Interrupt request gets triggered $\rightarrow$ (2) Stop current execution of program and save context $\rightarrow$ (3) Execute interrupt handler $\rightarrow$ (4) Once finished, reload context and continue.

8.4 Pulse Width Modulation

How can a μC communicate with other digital modules (sensors, actuators, display modules, ...)?

$\rightarrow$ For sensor and actuators, Pulse Width Modulation (PWM) is among the most frequent methods.

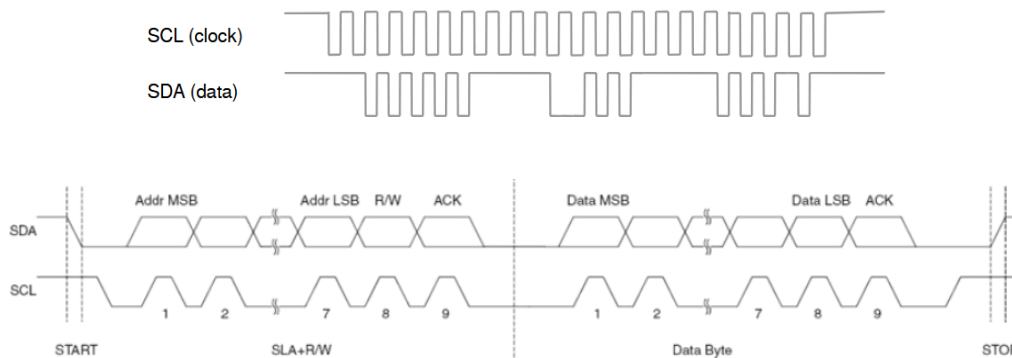
- PWM is a digital modulation method to represent a signal as a rectangular wave with varying duty cycle D (i.e. proportion of 'on' time over regular interval)
- PWM can control electrical signal power or amplitude, e.g. to switch power supply
- Low power loss: No current in 'off' signal state; no voltage drop over switch in 'on' state

Triangle Waveforms: Three types (lead, centre, trail), the name describes the location of the peak

Intersective PWM: Simple PWM generator principle with fixed period T and varying duty cycle. Uses a comparator to switch PWM output state (high, low) when input wave intersects with triangle wave.

8.5 Inter-Integrated Circuit (I²C)

I²C is a single-ended, synchronous, multi-controller/multi-target ("master-slave") serial bus. It has two lines: SCL (clock) and SDA (data).



8.6 Serial Peripheral Interface (SPI)

SPI is a full-duplex, synchronous, master-slave, serial communication bus with 4 lines: SS/CS (chip/slave select), SCLK (clock), MOSI/PICO (main data), and MISO/POCI (sub data).

While I²C has a max bandwidth of 5Mbit/s, SPI has an arbitrary bandwidth (no max clock speed). Data is transferred in 4-16 bits.

8.7 Testing and Debugging

Emulation

Imitate behaviour of target system as close as required, thus substituting the object that is emulated. The internal state of the emulation mechanism does not have to accurately reflect the internal state of the target as long as the behaviour matches.

Simulation

Evaluate with a model of the target system. Ideally, the simulation makes properties of the target system observable. **A simulation does not always lead to emulation.**

On-Target

Evaluate (behaviour) on actual target hardware. The evaluation is usually done in a lab or simulation environment to control external influences.

In-Field

Evaluate on actual target hardware within the targeted application environment.

JTAG

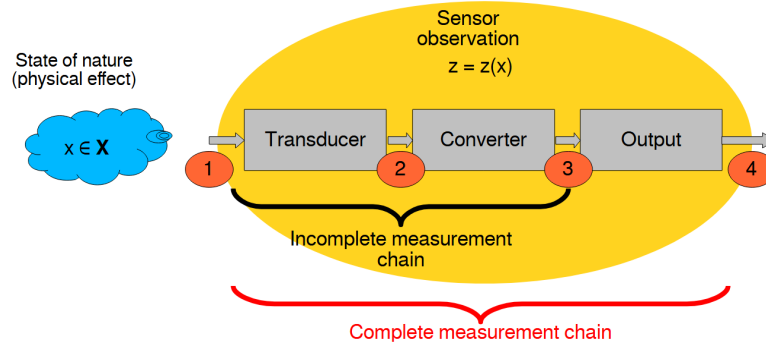
A boundary scan bus for testing and debugging integrated circuits. It features a serial protocol with 5 lines: TDI (test data in), TDO (test data out), TCK (test clock), TMS (test mode select), and TRST (test reset, optional). Typically operates at 10-100MHz.

JTAG can be used for design verification and debugging (controllability), automated manufacturing tests, commissioning (programming of non-volatile memories), and to hack devices (all without physical access to every PCB line). For authentication it uses chain encryption.

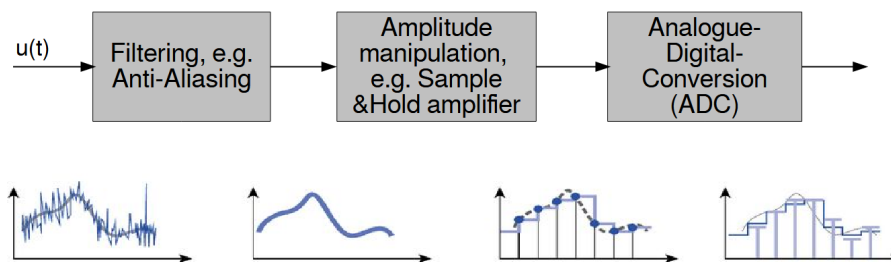
9 Sensors and Actuators

9.1 Measurements

Sensors model a state of the environment through (time discrete) observations.



The (analogue) signal captured by sensors is then digitised, i.e. converted from being continuous in t, x to being discontinuous in both dimensions. To achieve this, different conversion strategies exist, all differing in conversion speed, accuracy of the values, and amplitude resolution after quantisation.

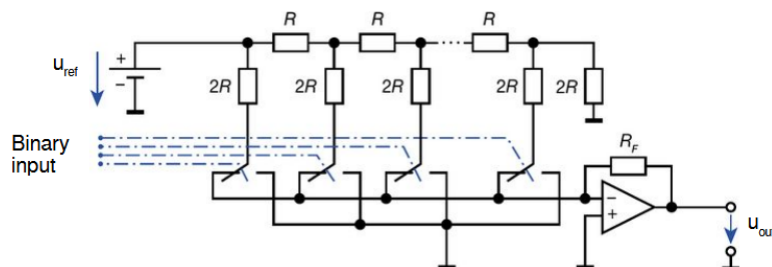
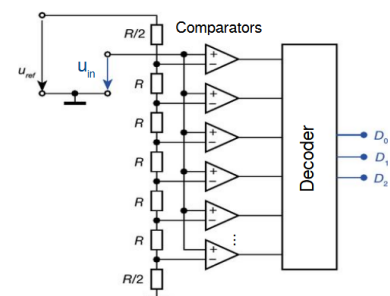


9.2 Parallel/Direct Conversion

Parallel converters work by comparing input voltage to a reference voltage using $2^n - 1$ parallel comparators.

The example on the right shows a 3-Bit converter. This approach is typically limited to $2^N = 2^8 = 256$ states and achieves frequencies greater than 1GHz.

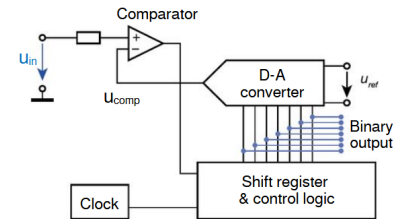
To convert an n-bit digital signal into analogue, one can then use a parallel DAC as pictured below (n switches). The problem is that digital switches could be asynchronous, causing undefined u_{out} .



9.3 Successive Approximation

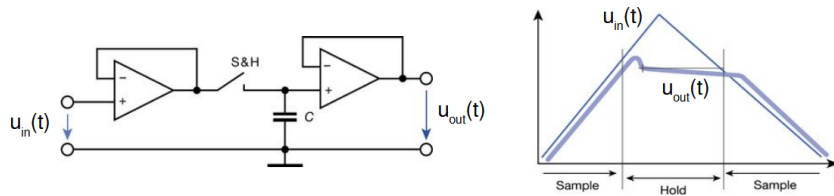
Successive Approximation is a sequential conversion technique that works according to the **weighting principle**.

A DAC creates different reference voltages to be compared to the input. SA can resolve 24bits and upwards at a frequency of around 1MHz.



9.4 Sample & Hold

The idea is to charge up to u_{in} as fast as possible (sample) and hold u_{out} constant for as long as possible.



9.5 Dual/Multi-Slope Integrating Converter

The main idea behind this technique is to convert an unknown voltage into a time delay, which is then encoded. It is accurate, produces low noise, has a linear transfer function, but is rather slow with a max. frequency of around 0.1-1kHz.

1. Integrate u_{in} for fixed time (until counter flips to 0), results in u_C
2. Apply u_{ref} with opposite polarity to integrator until 0V, thus comparator zeros and blocks clock at gate to maintain count result
3. Encode u_{in} as a function of u_{ref} , run-up time t_0 and run-down time t_U

9.6 Aliasing

Aliasing describes the effect when e.g. car wheels seem to spin backwards while accelerating. The same effect can also occur when sampling from an input signal.

To avoid aliasing, sampling intervals $T_s = 1/f_s$ must be carefully chosen so that they can represent the original signal. A signal with maximum frequency of f_{in} must be sampled at $f_s \geq 2f_{in}$. Aliasing occurs when $f_s < 2f_{in} \leftrightarrow 0.5f_s < f_{in}$.

For a signal with two frequency components, one must sample at $f_s \geq 2(f_b - f_a)$ to prevent aliasing.

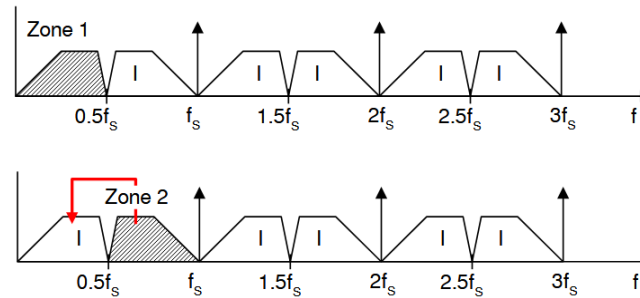
More generally, given a sampling rate f_s , two frequencies f and f' are aliases of each other if $\exists k \in \mathbb{N} : f' = f + kf_s$. The **Nyquist Theorem** states, that the only range in which the original frequency can correctly be sampled/reconstructed is the **Nyquist bandwidth** from 0 to $0.5f_s$.

Example: sampling at $0.2f_s \rightarrow f_s \geq 5f_{in} \rightarrow$ no aliasing.

9.7 Aliasing Filters

Baseband sampling: All signals of interest lie within Nyquist bandwidth (first Nyquist zone). Images at higher frequencies are filtered.

Undersampling: Sampling signals outside first Nyquist zone. Image in first Nyquist zone contains all information in frequency-reversed order. Sampled signal may lie in any Nyquist zone. Image in first Nyquist zone retains all information. For this the Nyquist criterion is updated to $f_s \geq 2\Delta f$ where Δf is the bandwidth of the signal of interest.



10 Processors and Controllers

10.1 Microprocessor

The microprocessor is *just* the computing unit and has to be complemented by different types of memory, I/O interfaces and other system components. They are usually fast but therefore expensive.

10.2 Microcontroller

A microcontroller (μC) is a System-on-Chip (SoC), which contains all basic components such as memory, timers, and various I/O interfaces (I²C, SPI, UART). In comparison to microprocessors they are slower, but also cheaper and more energy efficient.

Classically they are used, when control operations are more frequent than data intensive computing (e.g. in multimedia).

To use a μC , the ES needs to fulfill multiple efficiency requirements: Energy efficiency (especially when using batteries), memory efficiency (thus code and architecture efficiency), run-time efficiency (processing efficiency for real-time use critical)

11 Embedded AI

End devices at the end of a network are called edge devices. Running AI (EAI) directly on these devices can have some advantages, but is usually much more complex and requires adjustments for individual devices.

Advantages: Maximise real-time performance (inference), privacy, reduce bandwidth requirements, cost, no network delay, offline use

Drawbacks: Very limited compute power, training and deployment has to be tailored for device

11.1 Distributed Training

Centralised (level 1-3): Model is trained in the cloud, end devices collect data and send it to the cloud, the cloud returns the inference results (during inference)

Decentralised (level 5): Each device trains its own model locally (with local data), to obtain global model all devices need to share model updates via network, ideal for privacy

Hybrid (level 4-5): Combines centralised and decentralised approach (e.g. cloud can be used to additionally train on public data)

11.2 Model Compression & Optimisation

Since memory and efficiency are hard requirements in ES, the used models often need to be adjusted to fit onto the device (either before training when training on the device itself, or before deployment when training in the cloud).

11.2.1 Compression

Weight pruning: Removes redundant model weights (after training), requires post-finetuning to minimize accuracy reduction

Data quantisation: Replace number representation with lower-memory representation (e.g. 32-bit float to 16-bit float), reduces memory usage and accelerates computation

11.2.2 Optimisation

Model partitioning: Partition the model and distribute partitions between device and cloud; can be used to optimise latency, energy, and privacy

Early exit: Leverage output data of early layers (inference using partial model)

Edge caching: Optimises latency by caching latest model inference results at network edge

Model selection: Dynamically select DNN model from a set during inference